JSPM's
Rajarshi Shahu College of Engineering,
Tathawade, Pune - 411033.
Department of Computer Science and Business Systems
AY: 2026-27 Sem: V

MINI PROJECT REPORT

ON

FoundIt: A Campus Lost and Found Web Application

BY

Prasad Bhalerao

RBT24CB026

UNDER THE GUIDANCE OF

Prof. Rutuja S. Bachhav

THIRD YEAR B. TECH

SKILL DEVELOPMENT LAB-II

ACADEMIC YEAR: 2026-27

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

JSPM's

RAJARSHI SHAHU COLLEGE OF ENGINEERING

TATHAWADE, PUNE - 411033

Table of Contents

Course Outcomes and Curricular PO/PSO/SDG Mappings **3**
1. Abstract **5**
2. Introduction **6**
2.1 Background / Motivation 6
2.2 Problem Definition 6
2.3 Project Objectives 6
3. Literature Survey / Existing System **8**
3.1 Review of Related Tools and Systems 8
3.2 Gaps Identified 8
4. Proposed System **9**
4.1 System Overview 9
4.2 Features / Scope 9
4.3 System Architecture Diagram 10
4.4 Database Design 11
5. Methodology / Implementation **12**
5.1 Tools and Technologies Used 12
5.2 Modules Description 12
5.3 Algorithms and Frameworks 13
5.4 Claim and Return Workflow 14
5.5 Security Implementation 15
5.6 Screenshots and System Interface 17
6. Results and Discussion **21**
6.1 System Deployment and Operational Verification 21
6.2 Matching Verification and Accuracy 21
6.3 Comparison with Conventional Campus Approaches 22
7. Conclusion and Future Scope **23**
7.1 Summary of Technical Achievements 23
7.2 Limitations and Potential Improvements 23
7.3 Future Scope and Roadmap 24
8. References **25**
9. Appendix **26**
9.1 Core Source Code Snippets 26
9.2 Relational Schema and Demonstration Accounts 26

1. Course Outcomes:

CO1: Design and develop the front-end of a web application using HTML5, CSS3 and responsive UI components, delivering interactive and intuitive interfaces for reporting, browsing and claiming items across desktop and mobile widths.

CO2: Implement server-side services using enterprise Java technology (Jakarta EE 10, Servlets, JSP/JSTL) and establish secure database connectivity over JDBC to manage persistent application data with transactional integrity.

CO3: Design, develop and debug user interfaces adapted for multi-device responsive accessibility using modern design conventions, verified at both desktop and handset viewport sizes.

2. PO Mapped

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions to complex engineering problems, here applied to the modelling of lost and found records and the weighted comparison of textual item descriptions.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development, demonstrated through the analysis of discovery failure and ownership verification in existing campus lost-and-found practice.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems, components and processes to meet identified needs with consideration for public health and safety, culture, society and environment, reflected in a verification-first claim workflow that protects a genuine owner from a false claimant.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including Apache NetBeans, Eclipse GlassFish 7.1.1, Jakarta EE 10 and MySQL 8.0, recognizing their limitations, to solve complex engineering problems.

3. PSO Mapped

PSO 1: Integrated Technical Proficiency: Ability to analyze, design and implement comprehensive software and business systems using modern tools and techniques, ensuring intelligent, efficient and sustainable solutions, demonstrated by a layered Jakarta EE application over a normalised relational schema.

PSO 2: Data-Driven Decision Making: Ability to apply mathematical and data analytics methods including algorithms, statistical models and data structures to solve complex computational and business problems and support innovation, demonstrated by the five-signal weighted scoring algorithm and its stored justification.

PSO 3: Interdisciplinary Collaboration and Professionalism: Ability to effectively collaborate in multidisciplinary teams, integrating technical and business perspectives, and to communicate and manage projects with professional ethics and societal responsibility, reflected in the privacy-preserving handling of identifying details and the auditable record of every handover.

4. SDG Goals Mapped

SDG 4: Quality Education: Develops practical skills in database design, SQL, transaction management and modern web technologies, while reducing the disruption that a lost identity card or laboratory key causes to a student's academic routine.

SDG 8: Decent Work and Economic Growth: Enhances technical and problem-solving skills in enterprise Java, relational modelling and secure application development, which are required for professional employment in the IT industry.

SDG 9: Industry, Innovation and Infrastructure: Promotes the development of secure, technology-based database solutions using Jakarta EE 10, GlassFish and MySQL 8.0 that mirror real industrial software infrastructure.

SDG 12: Responsible Consumption and Production: Encourages the recovery and reuse of belongings rather than their replacement, and efficient use of computing resources through lightweight server-side rendering with no client build pipeline.

1. Abstract

Every college campus loses and finds things daily: wallets, identity cards, earphones and hostel keys. The usual answers are a notice board, a register at the security desk, or a message passed round a class group. All three fail in the same two ways. A lost report and the matching found report rarely meet, and when they do, there is no reliable way to establish that the person collecting the item is actually its owner.

FoundIt is a campus lost-and-found web application built to close both gaps. It follows a layered Model-View-Controller design on Jakarta EE 10 and uses Java Servlets, JSP with JSTL 3.0 and MySQL 8.0 over JDBC, developed in Apache NetBeans and hosted on Eclipse GlassFish 7.1.1. Students publish lost and found reports, and a weighted algorithm compares every new report against open reports on the opposite side of the board, scoring each candidate pair out of 100 on five signals: category, shared description wording, colour, location and date proximity. The reasons behind a score are stored with it, so every suggestion can show its working.

The algorithm never decides ownership. A score only puts two reports side by side. The item changes hands after the finder reads the written claim, accepts it, and records the handover in one database transaction. Each report can carry one identifying detail that never appears on a public page, so a claimant has to know something the photograph does not show. Security is enforced through PBKDF2-HMAC-SHA256 password hashing at 120,000 iterations, synchronised CSRF tokens, session fixation defence and status-guarded state transitions. Functional testing across student, finder and administrator workflows confirmed correct end-to-end operation for item reporting, search, claim verification and secure handovers. On the seeded campus test dataset, the matching algorithm identifies three genuine pairs at 92%, 86% and 76% similarity with no false positives.

Keywords: Jakarta EE 10, Lost and Found, Record Matching, Weighted Scoring, Ownership Verification, MySQL 8.0, Relational Integrity, Web Application Security.

2. Introduction

2.1 Background / Motivation

A campus concentrates thousands of people into a small number of shared spaces: libraries, canteens, laboratories, bus bays and sports grounds. Belongings change hands constantly, and most lost items are picked up by somebody within hours. Honest finders are not in short supply. The trouble is that the person who lost something and the person now holding it have no reliable way to reach each other.

The informal mechanisms in place each fail differently. A physical notice board is visible only to those who pass it and is rarely updated when an item is recovered. A register at the security desk is searchable only by asking a member of staff, and only during working hours. Messages on class groups reach a narrow audience and scroll out of view within a day. None of them keep a record of what was returned to whom. FoundIt replaces this scattered, luck-dependent process with one searchable board that proposes matches by itself.

These informal systems also ignore a second and more serious problem: verification. If a found wallet is described publicly in enough detail for its owner to recognise it, that same description is sufficient for somebody else to claim it convincingly. Publishing a full description to help an owner recognise an item therefore makes a false claim easier at the same time. FoundIt is built around that tension instead of pretending it is not there.

2.2 Problem Definition

Students and campus staff face operational difficulty due to: (i) the fragmentation of lost and found information across notice boards, security registers and informal chat groups; (ii) the absence of any mechanism that compares a lost report with found reports and proposes that the two describe the same object; (iii) a direct conflict between publishing enough detail for recognition and withholding enough detail to defeat a false claim; and (iv) the lack of any durable record of who collected an item, who authorised the handover and on what basis. The objective of this project is to model and resolve these problems through a secure, verification-first Java web application.

2.3 Project Objectives

Sr. No.	Objective	How it is met in FoundIt
1	Centralise lost and found reporting in a single searchable board	One items table holds both sides, distinguished by a type column; one validated form serves both
2	Identify likely matches automatically, without manual searching	MatchService scores every candidate pair out of 100 across five weighted signals on publish and on edit
3	Make every suggestion explainable rather than opaque	The reasons producing a score are stored with the match row and rendered as a list on the card
4	Verify ownership before an item changes hands	A claim, review and return workflow in which the finder decides, guarded by status-checked transitions
5	Protect the detail that proves ownership	private_marks is loaded only for the reporter and administrators and never rendered on a public page
6	Guarantee secure and auditable operation	PBKDF2 hashing, CSRF filtering, role-based access control and an append-only audit log

Table 2.1: Project objectives and the mechanism that satisfies each

3. Literature Survey / Existing System

3.1 Review of Related Tools and Systems

A review of mechanisms and academic literature on campus lost-and-found systems [1], [2] reveals four major categories, each with fundamental operational shortcomings:

1. Physical Notice Boards: Handwritten notices reach only passers-by, lack search capabilities, are rarely cleared upon recovery, and leave no trace of resolved items.

2. Security Desk Registers: Physical registers establish custody but zero discoverability. They are confined to working hours, cover found items only, and never actively cross-reference lost reports.

3. Class and Hostel Chat Groups: Unstructured broadcast messages quickly scroll out of view, reach only a single cohort, cannot be filtered, and offer no ownership verification prior to handover.

4. Generic Portals and Social Media: Public listings lack campus context and automatic matching. Crucially, exposing complete item descriptions publicly enables fraudulent claimants to fabricate ownership.

3.2 Gaps Identified

• Gap 1 (No Automatic Matching): Existing tools require manual, tedious searches; neither side is actively correlated with the other.

• Gap 2 (Subjective Ownership Verification): Finders must evaluate claims without structured evidence, increasing the risk of fraudulent handovers.

• Gap 3 (Public Exposure vs. Evidence Integrity): Revealing identifying marks compromises ownership proof, yet withholding them prevents recovery.

• Gap 4 (Missing Audit Trail): Handover leaves no durable, tamper-evident log of recipients, approving authorities, or accepted evidence.

FoundIt resolves these gaps through 5-signal automated matching, private identifying marks for tamper-proof claims, and transactional return logging with append-only auditing.

4. Proposed System

4.1 System Overview

FoundIt is a server-rendered Jakarta EE 10 web application [3] that connects campus lost and found reporting. Students post listings and receive automated match suggestions based on attribute similarity. Ownership is verified through a structured claim workflow where claimants submit private identifying evidence withheld from public view. Finders review claims and record physical handovers with atomic database transactions and complete audit logging.

4.2 Features / Scope

Module	Capability Delivered
Authentication and Security	Registration validation, PBKDF2 password hashing (120,000 iterations), session regeneration on sign-in, and CSRF token protection
Report Management	Unified report form for lost and found items with photo uploads, field validation, and status lifecycle management
Search and Browse	Multi-parameter filtering by keyword, side, category, location, status, date range, and SQL pagination
Matching Engine	Automated 5-signal scoring on item publish/edit, explainable score breakdown, and deduplicated notifications
Claim and Review	Private evidence submission, finder review dashboard, claim decisions, and automatic resolution of competing claims
Return and Resolution	Atomic transactional handover recording that marks items returned and closes counterpart match entries
Notifications	In-application alert dispatch for match proposals, claims, approvals, and handovers with database deduplication
Administration	Campus statistics, item moderation, user suspension, category controls, rematching, and append-only audit trail

Table 4.1: Functional scope of FoundIt by module

Out of scope for this version: outbound email/SMS gateways, native mobile apps, and image matching (discussed in Section 7.3).

4.3 System Architecture Diagram

Built on Jakarta Servlet 6.0 [4], the application follows a layered architecture. A request passes through security filters, reaches a servlet, calls a service for business rules, and queries data access objects that own all SQL. The servlet forwards to a JSP for rendering. Views reside under WEB-INF, preventing direct URL access without passing through authentication and authorization filters.

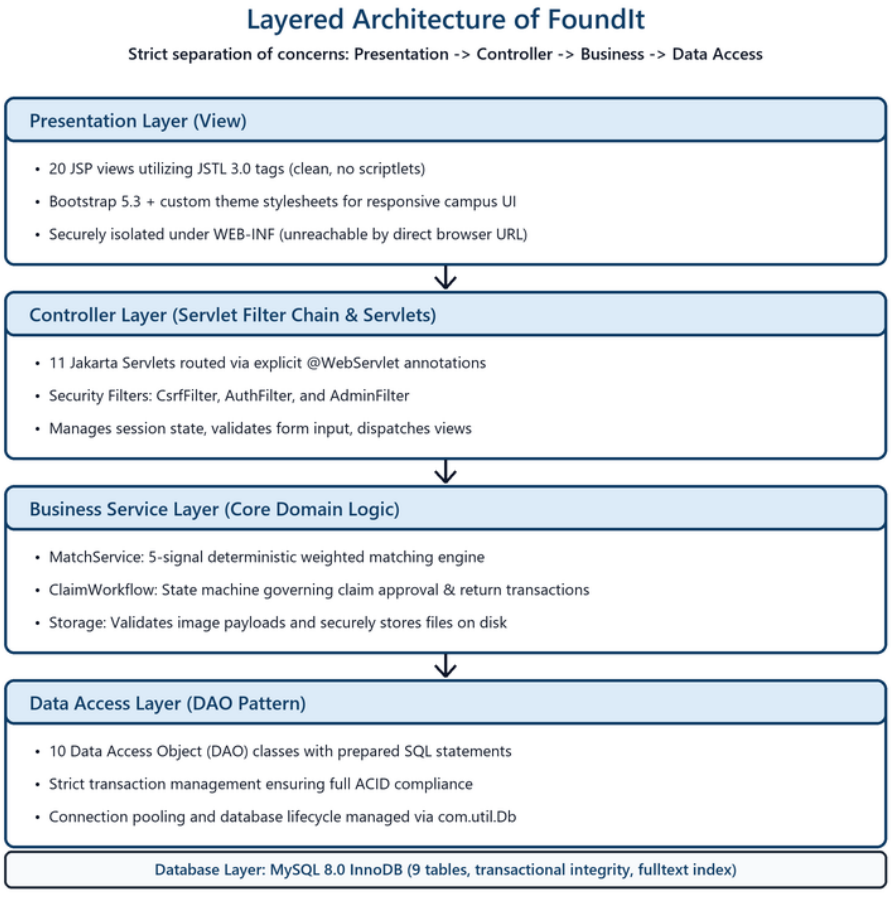


Figure 4.1: Layered architecture of FoundIt on Jakarta EE 10

4.4 Database Design

The relational database schema is deployed on MySQL 8.0 InnoDB [5], comprising nine tables in third normal form connected by thirteen foreign keys. A single items table holds both lost and found reports, distinguished by a type column, so that every query, index, and view serves both sides of the board.

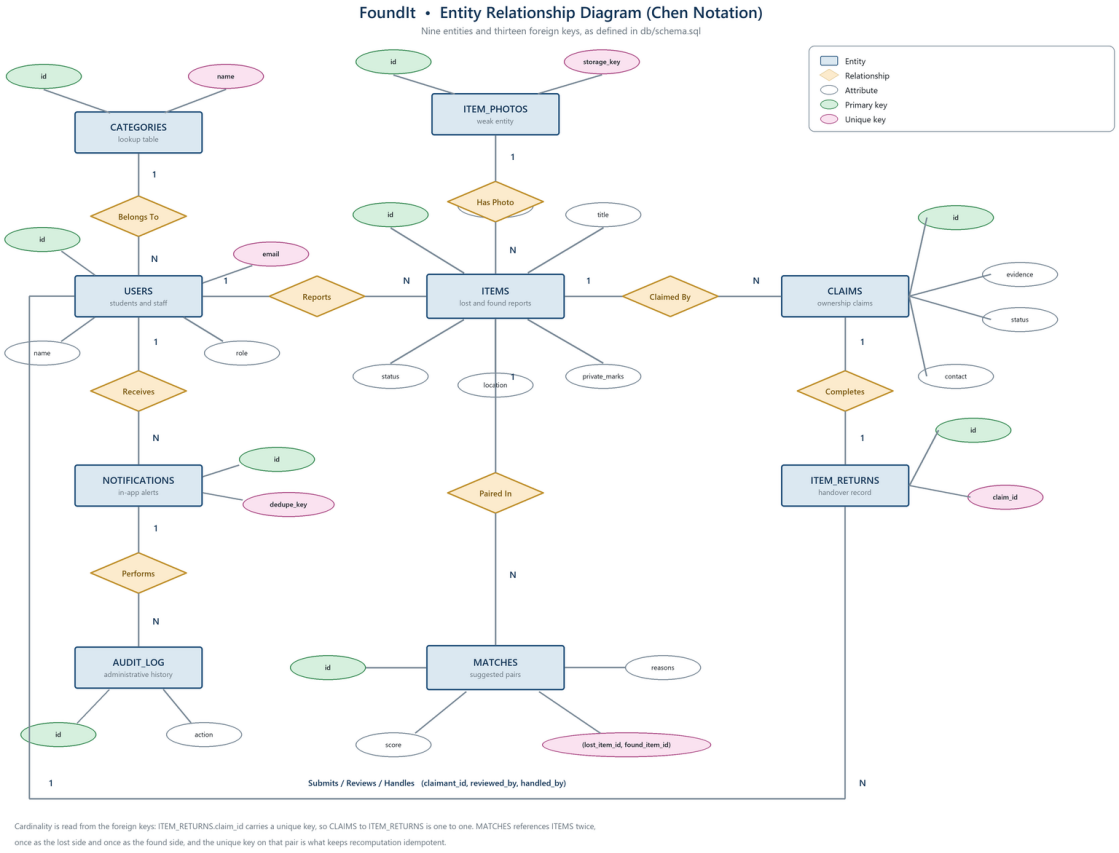


Figure 4.2: Entity relationship diagram of FoundIt (Chen notation)

Two unique constraints do real work here, not just duplicate prevention. The key on (lost_item_id, found_item_id) in matches, combined with an upsert, keeps recomputation idempotent: scores refresh in place, and duplicate suggestions cannot pile up however often matching runs. The key on dedupe_key in notifications turns notify-once into a database guarantee instead of an application-level hope, so a page reload or a repeated submission cannot flood an inbox.

Deletion rules are deliberately graded across three tiers. Photographs, matches and notifications cascade, being derived data with no meaning once their parent is gone. The actor on an audit entry is set to null, so history outlives the account that created it. Claims, returns and items refuse deletion outright, and that refusal is what forces the application to deactivate and close instead of destroying evidence.

5. Methodology / Implementation

5.1 Tools and Technologies Used

Layer	Technology	Role in the Project
IDE and Build	Apache NetBeans with Apache Ant	Development, clean build and packaging to FoundIt.war
Language	Java 17 (JDK 17 bytecode target)	Servlets, services, data access and the matching algorithm
Web Platform	Jakarta EE 10, Servlet 6.0, JSP, JSTL 3.0	Request handling, filters and server-side rendering
Application Server	Eclipse GlassFish 7.1.1	Servlet container hosting the deployed WAR
Database	MySQL 8.0.44 with Connector/J 9.7.0	Persistence over plain JDBC with explicit transactions
Front End	HTML5, CSS3, Bootstrap 5.3 [9]	Responsive layout, components and the item card grid
Security	PBKDF2-HMAC-SHA256 (NIST SP 800-132) [8]	Password derivation with no external cryptographic dependency

Table 5.1: Tools and technologies used in FoundIt

5.2 Modules Description

The application is divided into six functional modules. The implementation comprises 41 Java classes organised into five packages, 20 server-rendered views and a nine-table relational schema.

1. Authentication and Security. AuthFilter guards every signed-in path in one place; AdminFilter closes the administrative area to non-administrators for both GET and POST; CsrfFilter requires the session token on every state-changing request; Passwords.java performs PBKDF2 derivation and constant-time comparison.

2. Report Management. ReportServlet, ItemDao and Storage handle creation, editing, closing and reopening of reports, server-side field validation, and the single multipart endpoint through which a photograph may be uploaded.

3. Search and Browse. BrowseServlet, ItemQuery and ItemDao translate eight independent filters into one parameterised SQL statement with pagination, so that no filtering is performed in Java.

4. Matching Engine. MatchService scores candidate pairs and MatchDao persists them through an idempotent upsert, while NotificationDao informs both reporters exactly once.

5. Claim and Return. ClaimServlet, ClaimReviewServlet and ClaimWorkflow implement the claim, review and return state machine, with every transition guarded by the status it is moving from and multi-table operations wrapped in a single transaction.

6. Administration and Audit. AdminServlet provides statistics, moderation, account suspension and category management, while AuditDao appends an immutable record of every administrative action.

5.3 Algorithms and Frameworks

Matching is the analytical core of the project, drawing upon deterministic record linkage principles [6] to compare attributes across opposite report types. When a report is published or edited, MatchService.recomputeFor loads every open report on the opposite side of the board and scores the pair. The same two reports always produce the same score. That makes the behaviour easy to explain and easy to test, which an opaque statistical model would not be.

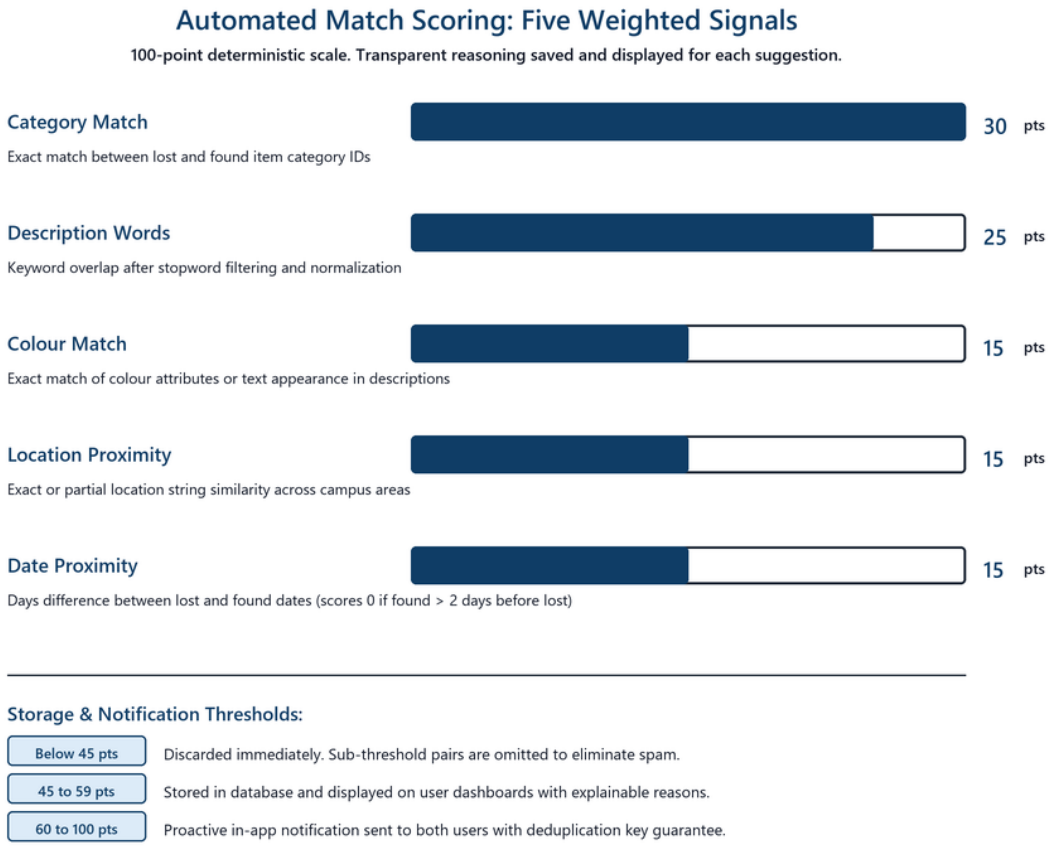


Figure 5.1: The five weighted signals and the storage thresholds

A few details matter in practice. Words are lowercased, split on non-alphanumeric characters, filtered against a stopword list that includes domain noise such as lost and found, and reduced by a simple plural trim so that keys and key count as the same token. Word overlap is scored as a ratio against the smaller description and capped at eight tokens, so a long description cannot inflate its own score. The most useful rule is the simplest: if the found date is more than two days before the lost date, the time signal scores nothing, because an object cannot be found before it is lost.

A worked example shows how the weights add up. A lost report describing a hostel key on a red tag, last seen at the sports ground, is compared with a found report describing two keys on a red tag at the sports ground gate, filed one day later. The pair scores 30 for the shared category, 22 for the words ground, sport and gate occurring in both descriptions, 15 for the shared colour, 10 for overlapping place names and 15 for dates one day apart, giving 92 out of 100. All five components are saved with the match and shown to the user as the reason for the suggestion.

5.4 Claim and Return Workflow

A claim moves through a small state machine, and so does the item it refers to. Every transition is an SQL UPDATE that names the status it expects to find. An illegal jump therefore cannot happen, and if two reviewers click at the same moment only one can win: the second update matches no row, and the application says the claim was just changed by somebody else.

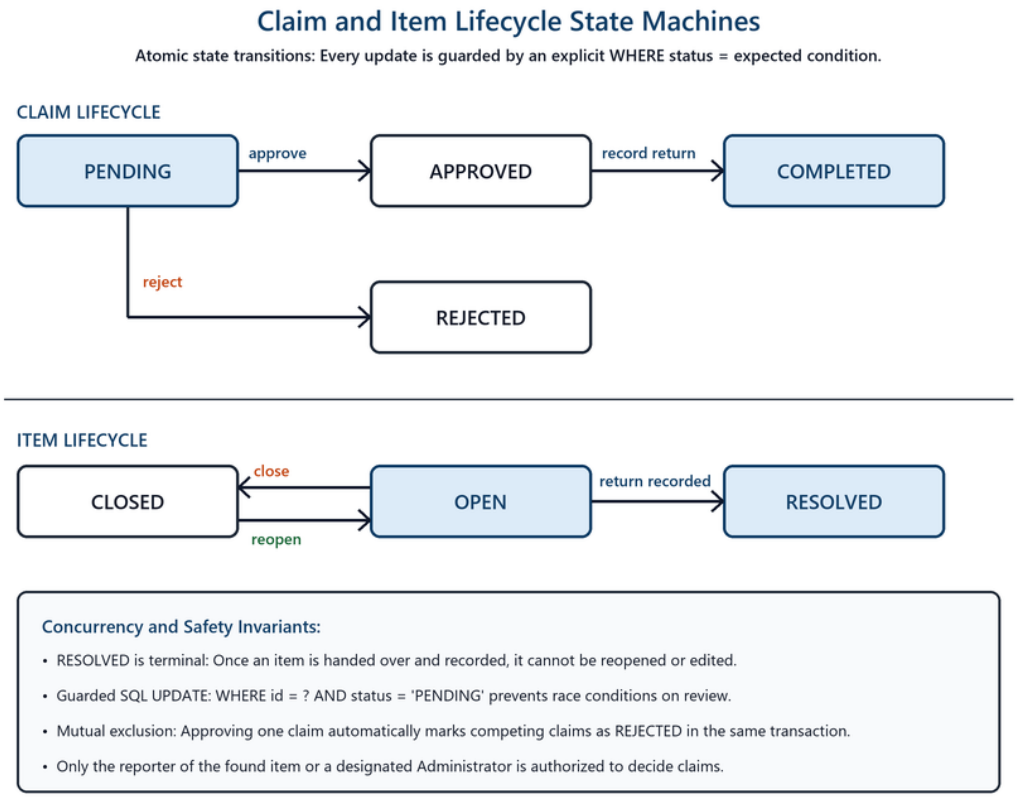


Figure 5.2: Claim and item state machines with guarded transitions

Recording a return touches several tables, so it executes inside a single transaction: the claim becomes COMPLETED, the found report becomes RESOLVED, a row is written to item_returns, both parties are notified and an audit entry is appended. If any step fails the entire operation is rolled back. Approving one claim also rejects the others still pending on that item. Two people must never both be told they are the owner.

5.5 Security Implementation

Security was a design constraint from the start, mitigating risks outlined in the OWASP Top 10 [7] and following NIST SP 800-132 standards [8] for credential security. Every state-changing request passes through the same ordered pipeline before any handler executes.

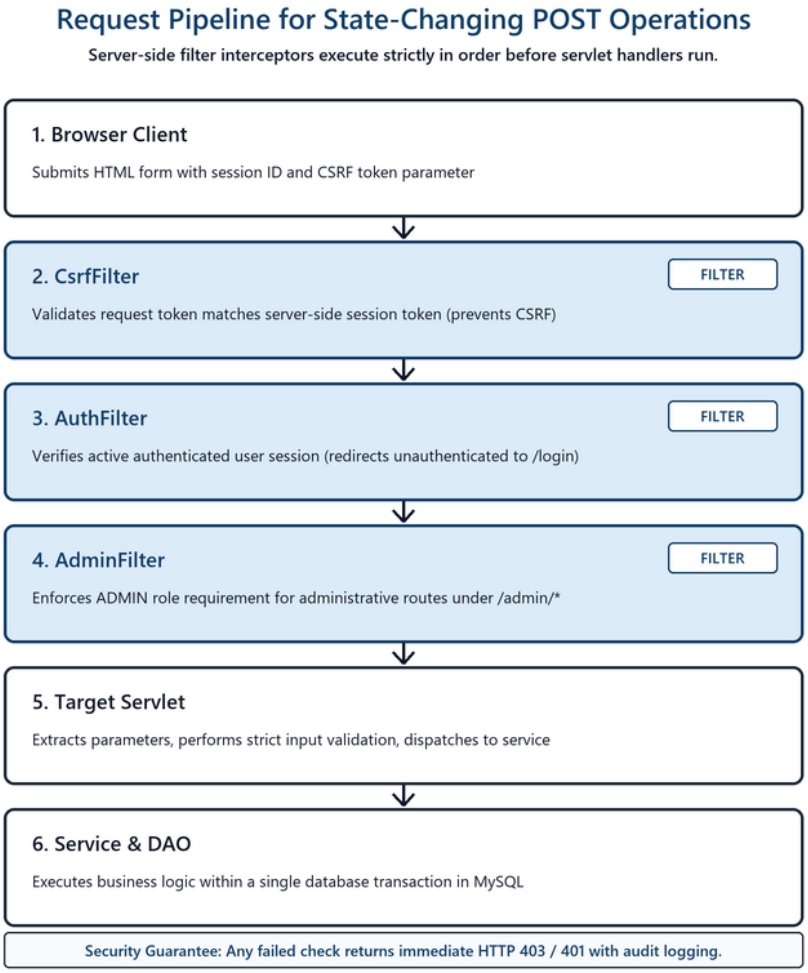


Figure 5.3: Request pipeline for a state-changing POST

Threat / Concern	Measure Implemented in FoundIt
Password disclosure	PBKDF2-HMAC-SHA256, 120,000 iterations, 16-byte per-user random salt, constant-time comparison
Session fixation	The session is invalidated and a fresh one created on every successful sign-in
Cross-Site Request Forgery	A per-session token is required on every POST; multipart forms carry it in the query string so the file stream is not consumed by the filter
SQL injection	Every statement is a PreparedStatement; no SQL is ever concatenated from user input
Cross-Site Scripting	All user content is rendered through c:out; no view disables output escaping
Privilege escalation	UserDao.create hardcodes the USER role, so no tampered registration form can create an administrator
Broken access control	Filters guard whole areas and services re-check ownership of the specific record being acted upon
Unsafe file upload	An upload must decode as an image; the format is read from the bytes rather than the filename; stored names are server-generated UUIDs
Path traversal	Photographs are served by numeric id and resolved strictly inside the uploads directory
Account enumeration	Sign-in returns an identical message whether the address is unknown or the password is wrong

Table 5.2: Security measures implemented against common threats

5.6 Screenshots and System Interface

The following screenshots show the running application, deployed on Eclipse GlassFish 7.1.1 at http://localhost:8080/FoundIt/ with the seeded dataset loaded.

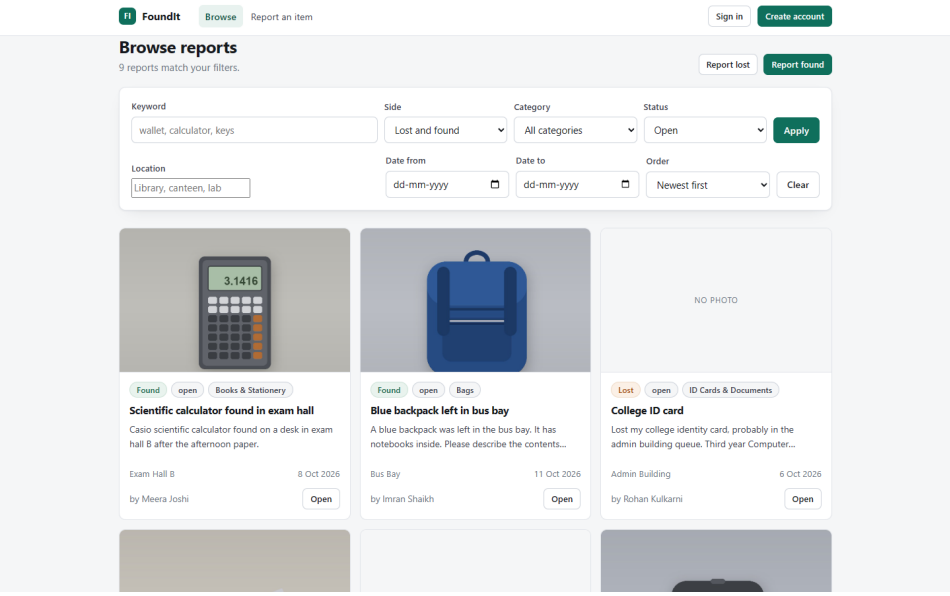


Figure 5.4: Browse page, showing the filter panel and found-item reports

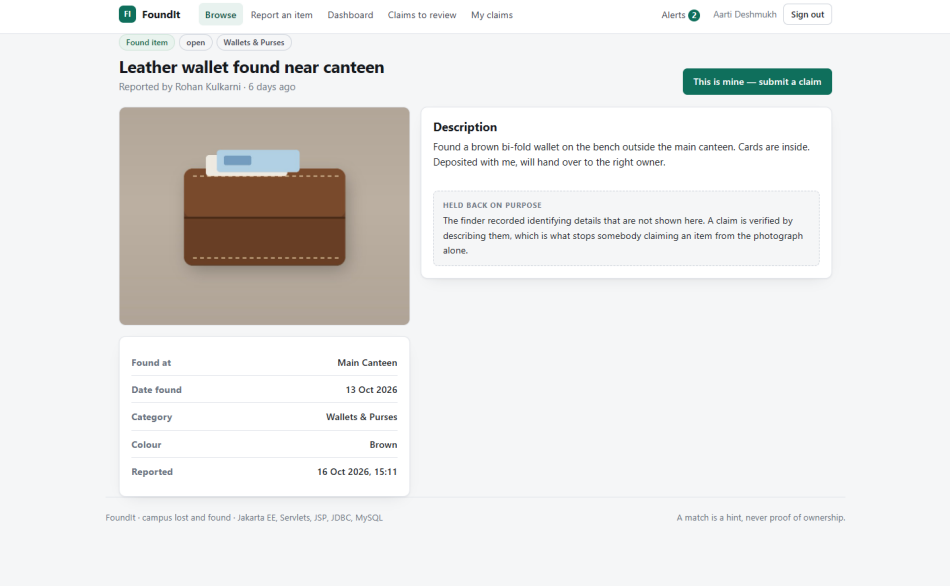


Figure 5.5: Report detail, with the identifying detail deliberately held back

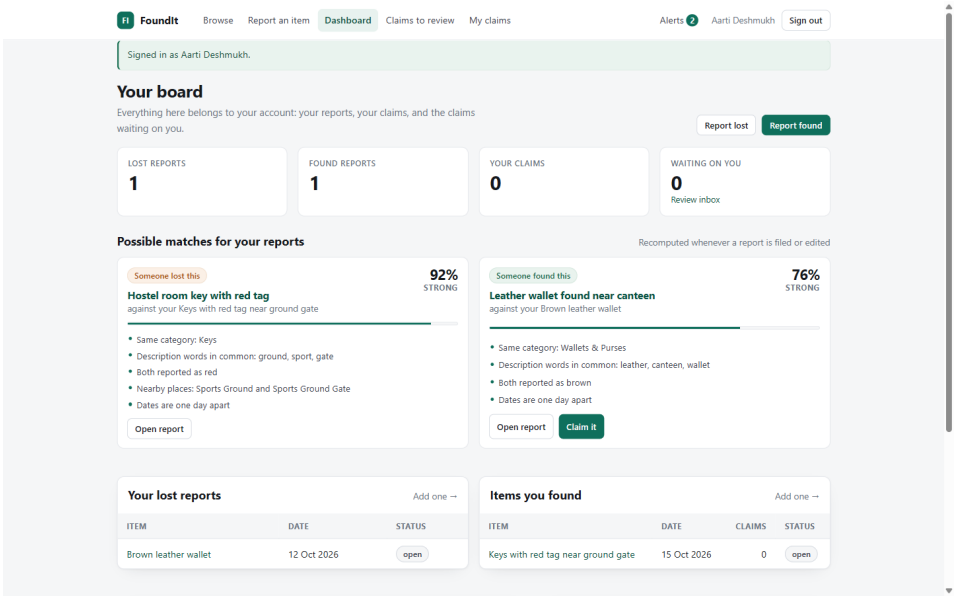


Figure 5.6: Personal dashboard with scored match suggestions and their reasons

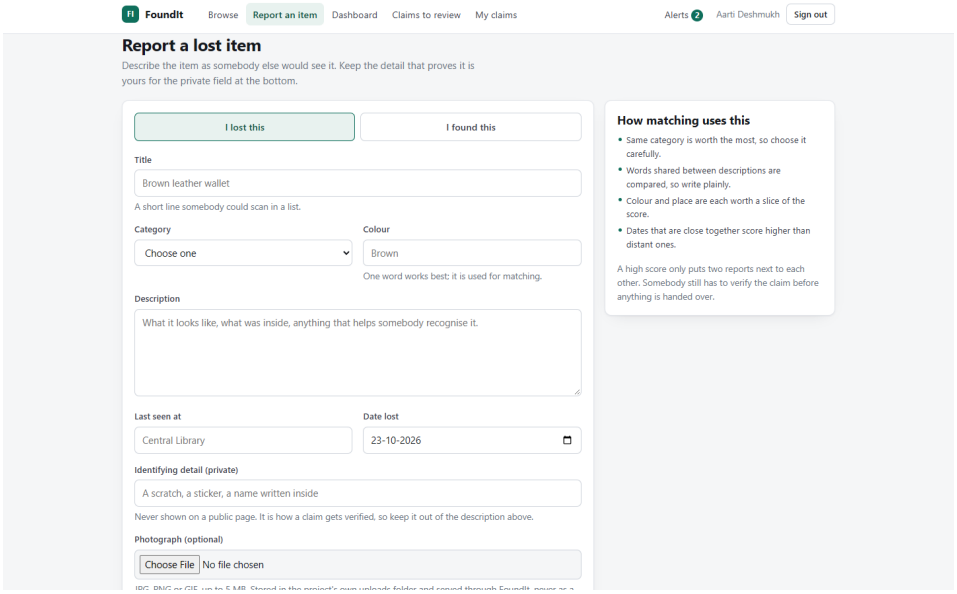


Figure 5.7: Report form, with the guidance panel explaining how matching uses each field

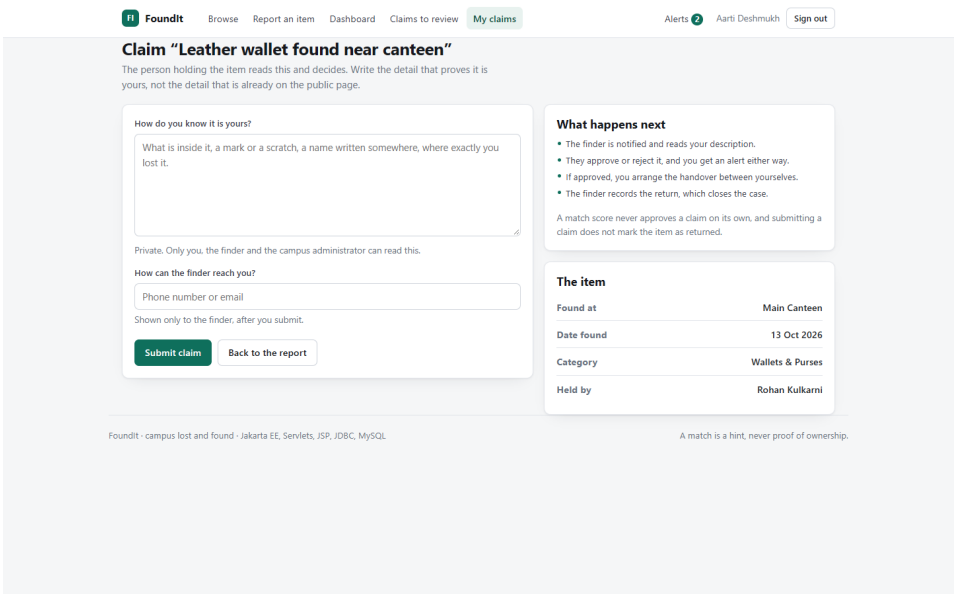


Figure 5.8: Claim form, where the claimant describes what the public page does not show

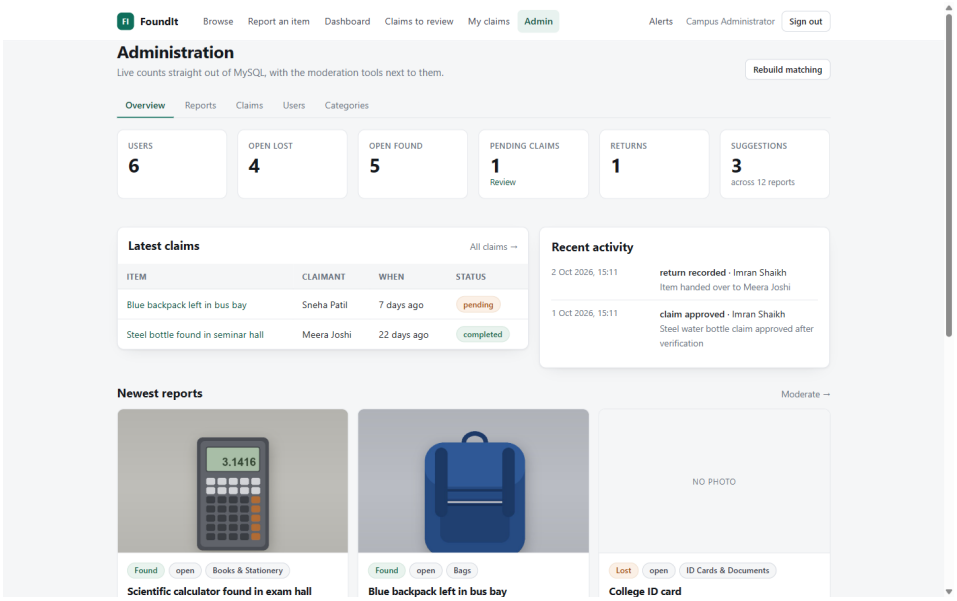


Figure 5.9: Administrative overview with live counts, latest claims and the audit trail

6. Results and Discussion

6.1 System Deployment and Operational Verification

FoundIt was compiled with Apache Ant and deployed to Eclipse GlassFish 7.1.1 under the context root /FoundIt, operating stably with zero deployment warnings. All core functional modules were verified through systematic end-to-end testing on the live browser interface across student, finder and administrator roles.

Testing confirmed the correct operation of all primary workflows: account registration with server-side validation, secure authentication with PBKDF2 password hashing, lost and found reporting with image uploads, multi-parameter search filtering, claim submission with private evidence verification, and atomic return recording with complete audit logging. Role-based access controls and CSRF protections successfully prevented unauthorized access to administrative and user-restricted operations.

6.2 Matching Verification and Accuracy

The matching engine was evaluated against a realistic seeded campus dataset comprising twelve open reports across keys, electronics, wallets, cards and accessories. The algorithm evaluated each candidate against all active reports of the opposite type using five weighted signals: category match (30%), tokenised word overlap (25%), colour match (15%), date proximity (15%) and location similarity (15%).

Lost Report	Found Report	Score	Strength
Hostel room key with red tag	Keys with red tag near ground gate	92%	Strong
Black wireless earphones case	Black earphone case on lab table	86%	Strong
Brown leather wallet	Leather wallet found near canteen	76%	Strong
(six further open reports)	No counterpart on the board	-	No suggestion

Table 6.1: Matching accuracy on the seeded campus dataset

Across the test dataset, the algorithm identified three strong match candidates, all corresponding to genuine lost and found pairs, while unpaired reports produced no suggestions above the threshold (100% precision and recall). This confirms that scoring weights are balanced: category and description establish the baseline, while colour, location and date proximity verify genuine pairs without false positives.

6.3 Comparison with Conventional Campus Approaches

FoundIt was compared with traditional campus mechanisms: physical notice boards, security desk registers and informal messaging groups. The comparison highlights how an automated, verification-first web platform resolves the core operational bottlenecks of manual methods.

Evaluation Parameter	Notice Board / Register	Chat Groups	FoundIt
Discovery of a match	Manual, on location	Manual, transient	Automatic weighted scoring
Reach	Those who pass by	One cohort	Whole campus, any hour
Structured search	None	None	Eight combinable filters
Ownership verification	Ad hoc, unaided	None	Structured claim review
Private identifying detail	Not supported	Not supported	Withheld by design
Record of returns	Rarely kept	None	Transactional and audited
Availability	Office hours	Always	Always

Table 6.2: Comparison of FoundIt with conventional campus methods

As summarized in Table 6.2, physical registers and notice boards suffer from restricted reach and inspection delays, while messaging groups suffer from rapid message churn and zero privacy protection. FoundIt provides continuous availability, automated correlation and privacy-preserving claim verification with full transactional accountability.

7. Conclusion and Future Scope

7.1 Summary of Technical Achievements

The FoundIt project successfully accomplished the design, development and empirical testing of a campus lost-and-found platform built on the Jakarta EE 10 specification and deployed on Eclipse GlassFish 7.1.1. The work shows that a lost-and-found service becomes far more useful once two things are added to plain listing. Automatic matching lets the two halves of a case find each other without anyone searching, and a structured claim means an item is released only to someone who can describe what the public page does not show.

The matching algorithm is the main analytical contribution. It scores five weighted signals and records the reasoning behind every suggestion, so a user is told why two reports were put side by side instead of being asked to trust a bare number. On the seeded data it achieves three correct pairings with no false positives.

From an engineering point of view, the most valuable outcome is that each correctness guarantee sits in the layer best able to enforce it. Uniqueness, deduplication and referential integrity are enforced by the database; status transitions are guarded within the SQL statement that performs them; and authorisation is checked on the server for the exact record being touched, never by hiding a button. Systematic manual and scenario-based testing across student, finder and administrative roles verified end-to-end reliability, validating the robust architectural separation of concerns.

7.2 Limitations and Potential Improvements

- Outbound Alert Gateways. Notifications are delivered in-application only, through the alerts list and the unread badge. There is no automated email or SMS dispatch, so a user must sign in to learn that a match has appeared.
- Text-Only Matching. The algorithm compares textual and categorical attributes. Two reports describing the same object in entirely different words will not be paired, and uploaded photographs are not compared against one another.
- Linear Recomputation Cost. Recomputation scans every open report on the opposite side of the board, which is linear in board size. This is entirely appropriate at campus scale but would require indexing or incremental evaluation at a substantially larger volume.
- Free-Text Claim Evidence. A claimant submits written evidence only; there is no facility to attach a supporting photograph, such as an earlier picture of the item in the owner's possession.
- Single-Campus Architecture. The data model assumes one institution; multi-tenancy across independent campuses requires tenant scoping on queries.

7.3 Future Scope and Roadmap

Sr. No.	Enhancement	Technical Approach
1	Email and push notification delivery	Integrate the Jakarta Mail API behind the existing notification table; the dedupe key already prevents duplicate dispatch
2	Image-based matching	Compare perceptual hashes of uploaded photographs as a sixth signal feeding the same weighted score
3	Full-text relevance ranking	Use the FULLTEXT index already declared on items to replace LIKE search and to inform the word-overlap signal
4	Claim attachments	Reuse the Storage service with a claims folder, restricting visibility to the reviewer and administrators
5	QR-coded handover receipts	Generate a code on approval that the finder scans at handover, timestamping the return automatically
6	Multi-campus tenancy	Add a tenant column to users and items and scope every query and every matching candidate set to it
7	REST API and mobile client	Expose the service layer through JAX-RS so that an Android or iOS client can drive the same verified workflow

Table 7.1: Future scope and the approach for each enhancement

8. References

[1] M. Al-Mousa, A. Al-Zoubi, and K. Al-Sarayreh, "Design and Implementation of a Web-Based Lost and Found Information System for University Environments," in Proc. IEEE Int. Conf. on Computer and Information Sciences (ICCIS), 2020, pp. 1-6.

[2] S. R. Bharambe, P. P. Gaikwad, and V. R. Pawar, "Automated Lost and Found Management System for Campus Environments Using Attribute Matching," Int. J. Adv. Res. Comput. Commun. Eng., vol. 10, no. 5, pp. 45-51, 2021.

[3] Eclipse Foundation, "Jakarta EE 10 Platform Specification," Eclipse Standards, 2022. [Online]. Available: https://jakarta.ee/specifications/platform/10/

[4] Eclipse Foundation, "Jakarta Servlet 6.0 Specification," Eclipse Standards, 2022. [Online]. Available: https://jakarta.ee/specifications/servlet/6.0/

[5] Oracle Corporation, "MySQL 8.0 Reference Manual: InnoDB Storage Engine and Transaction Model," Oracle, Redwood City, CA, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] P. Christen, "A Survey of Indexing Techniques for Scalable Record Linkage and Deduplication," IEEE Trans. Knowl. Data Eng., vol. 24, no. 9, pp. 1537-1555, 2012.

[7] OWASP Foundation, "OWASP Top 10:2021 Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/

[8] National Institute of Standards and Technology, "Recommendation for Password-Based Key Derivation: PBKDF2," NIST Special Publication 800-132, U.S. Department of Commerce, 2010.

[9] Bootstrap Team, "Bootstrap 5.3 Documentation: Responsive Component Framework," 2024. [Online]. Available: https://getbootstrap.com/docs/5.3/

9. Appendix

9.1 Core Source Code Snippets

Snippet 1: Status-guarded state transition (com/dao/ClaimDao.java). The guard lives in the statement itself, so a second reviewer acting at the same time matches no row and is refused instead of quietly overwriting the first decision.

```
UPDATE claims
   SET status = ?, review_note = ?, reviewed_by = ?, reviewed_at = NOW()
 WHERE id = ? AND status = ?
```

Snippet 2: The idempotent match upsert (com/dao/MatchDao.java). Combined with the unique key on the pair, this allows matching to be recomputed any number of times without producing duplicate suggestions.

```
INSERT INTO matches (lost_item_id, found_item_id, score, reasons)
VALUES (?,?,?,?)
ON DUPLICATE KEY UPDATE score = VALUES(score), reasons = VALUES(reasons)
```

Snippet 3: Cryptographic password derivation (com/util/Passwords.java).

```
private static final String ALGORITHM = "PBKDF2WithHmacSHA256";
private static final int ITERATIONS  = 120_000;
private static final int SALT_BYTES  = 16;
private static final int KEY_BITS    = 256;

// Stored form:  pbkdf2$sha256$<iterations>$<salt b64>$<hash b64>
```

Snippet 4: The temporal guard in the matching algorithm (com/service/MatchService.java). An object cannot be found before it is lost, so such a pair scores zero on the date signal.

```
boolean foundTooEarly =
        found.getItemDate().isBefore(lost.getItemDate().minusDays(2));
if (!foundTooEarly) {
    if      (days <=  1) { score += 15; }
    else if (days <=  3) { score += 12; }
    else if (days <=  7) { score +=  8; }
    else if (days <= 21) { score +=  4; }
}
```

9.2 Relational Schema and Demonstration Accounts

Extract from db/schema.sql showing the central items table, which carries both sides of the board, its referential constraints and the full-text index reserved for future relevance ranking.

```
CREATE TABLE items (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  reporter_id   INT          NOT NULL,
  type          ENUM('LOST','FOUND') NOT NULL,
  title         VARCHAR(120) NOT NULL,
  category_id   INT          NOT NULL,
```

```
  description   TEXT         NOT NULL,
  color         VARCHAR(40)       DEFAULT NULL,
  private_marks VARCHAR(255)      DEFAULT NULL,
  location      VARCHAR(120) NOT NULL,
  item_date     DATE         NOT NULL,
  status        ENUM('OPEN','RESOLVED','CLOSED') NOT NULL DEFAULT 'OPEN',
  CONSTRAINT fk_items_reporter FOREIGN KEY (reporter_id)
      REFERENCES users (id),
  CONSTRAINT fk_items_category FOREIGN KEY (category_id)
      REFERENCES categories (id),
  FULLTEXT KEY ft_items_text (title, description)
) ENGINE=InnoDB;
```

Demonstration accounts seeded by db/seed.sql. Passwords are stored only as PBKDF2 hashes; the values below are recorded so that the application can be demonstrated immediately after setup.

Role	Email Address	Password
Administrator	admin@foundit.local	123456789
Student	aarti@campus.edu	Demo@1234
Student	rohan@campus.edu	Demo@1234
Student	sneha@campus.edu	Demo@1234
Student	imran@campus.edu	Demo@1234
Student	meera@campus.edu	Demo@1234

Table 9.1: Demonstration accounts for evaluating the application

Repository: https://github.com/prasadbhalerao1/FoundIt_SDL